Devpost submission: AutoFixAI

Submission fields

Field	Value
Project name	AutoFixAI
Tagline	Red build in, verified pull request out: an autonomous CI-repair agent powered by NVIDIA Nemotron on Nebius Token Factory.
Track	Coding and Agentic Engineering
Demo URL	https://port8000-dptwsrmgxt3w28t.tunnel.applications.eu-south1.nebius.cloud
Code repository	https://github.com/Gaurav047/AutoFixAI (MIT license)
Video	https://youtu.be/lTOj6GO26-8
Existed before the submission period?	No. Built from an empty repository during the hackathon.
Built with	nvidia-nemotron, nebius-token-factory, nebius-serverless-endpoints, nebius-container-registry, python, fastapi, react, typescript, vite, docker, github-actions, openai-sdk, tavily

---

Project description

Paste everything below this line into the Devpost "About the project" field.

Inspiration

A red CI build blocks a whole team. Someone has to open a 400-line log, work out which line matters, check out the branch, reproduce the failure, find the regression and push a fix. Most of these failures are small: a half-finished refactor, a renamed field, a rounding change, an off-by-one. Each one still costs 20–60 minutes and breaks someone's focus.

Coding assistants can already write a patch. What they don't do is **own the loop**: reproduce the failure, fix it, *prove* the fix works, and hand over something a reviewer can merge. We wanted an agent that does exactly that, and that can't game the result.

What it does

AutoFixAI takes a repository with failing tests and returns a **verified pull request**:

1. **Reproduce.** Clones the repo into an isolated workspace and runs the real test command.

2. **Triage. Nemotron 3 Nano** classifies the failure, names the files it suspects and estimates difficulty, in about a second.
3. **Research.** When the failure points outside the repository (a removed API, a library or language-version change), **Tavily** searches the web and the findings go to the coding agent.
4. **Fix. Nemotron 3 Super** works in a tool-calling loop: it reads files, searches the code, makes exact edits and reruns the tests.
5. **Verify.** When the agent says it's done, AutoFixAI reruns the test suite **itself**. It rejects the fix if the tests fail or if any test file was touched.
6. **Reflect and escalate.** If an attempt is rejected, Nano writes up what went wrong, the workspace is reset to a clean tree, and **Nemotron 3 Ultra** takes the next attempt.
7. **Ship.** Nano writes a pull request with the root cause, the fix and the verification, plus the sources Tavily found. AutoFixAI opens it on a new `autofix/*` branch.

You can use it four ways:

- a **live dashboard** with a streaming agent timeline, diff viewer and per-model token usage
- a **CLI** (`python -m autofix fix <repo> --test "pytest -q" --pr`)
- a **drop-in GitHub Action** that runs whenever CI fails
- a **GitHub webhook** for a hosted setup

How we built it

Model routing across three Nemotron tiers, all served through **Nebius Token Factory's** OpenAI-compatible API:

Tier	Job	Why
Nemotron 3 Nano	Triage, reflection, PR write-up	Fast, cheap, structured output
Nemotron 3 Super	The coding agent	Reliable tool calling and code reasoning at a cost that allows multi-step loops
Nemotron 3 Ultra	Escalation only	Deep reasoning, used only where it's needed

A typical fix uses about **2k Nano tokens and 17–32k Super tokens, with no Ultra calls at all**. The dashboard shows this per run.

Agent design

- The agent uses native function calling, with seven tools: `list_files`, `read_file`, `search`, `edit_file`, `write_file`, `run_tests` and `finish`. With a Tavily key it gets an eighth, `web_search`, for its own lookups mid-fix.
- Edits are exact, unique search-and-replace operations. There's no fuzzy patching that could silently change the wrong code.
- Model IDs aren't hard-coded. AutoFixAI asks Token Factory's `/v1/models` endpoint what's available and picks the right Nemotron for each tier automatically.

- Reasoning is controlled per call. Nano's structured jobs run with thinking turned off, which is about 6× faster and avoids truncated JSON. Super and Ultra keep reasoning on for the fix loop.

Web research with Tavily. Some failures can't be understood from the repository alone, so AutoFixAI uses Tavily to research them:

- **Triage decides when to search.** Nano's triage includes a `search_query` when the fix depends on facts outside the repo. AutoFixAI only searches for failures categorised as import errors, dependency or API changes, or config problems, so logic bugs never trigger a search.
- **Findings go to the coding agent as background.** Tavily's answer and top sources are added to Super's starting context, and the agent can run further searches through `web_search`.
- **Every PR cites its sources.** The References section lists the URLs Tavily returned. AutoFixAI builds the list from the search results, not from model output, so it can't contain invented links.
- **It's visible.** The dashboard shows a Web research card (the query, Tavily's answer and the source links) and counts Tavily searches next to the model usage.

Guardrails. The agent should be safe to point at a real repository:

- Every path is confined to the run's workspace, with no `..` escapes and no `.git` access.
- The only command the agent can run is the project's test command.
- **Test files are protected.** Edits to tests are blocked when they're attempted and checked again on the final diff, so the agent can't make CI pass by deleting assertions.
- A rejected attempt starts again from a clean tree. The next attempt gets only the *lessons* from the last one, not its half-applied edits.
- AutoFixAI only ever opens PRs, never pushes to your branch, and ignores failures on its own branches so it can't loop.

Stack and deployment

- **Backend:** Python and FastAPI, with server-sent events for live progress.
- **Frontend:** React and TypeScript, built with Vite.
- **Deployment:** a Docker image in **Nebius Container Registry**, served from a **Nebius Serverless Endpoint**, in a public mode that only allows the bundled demo repos.
- **Tests:** an offline end-to-end suite that drives the *real* pipeline with a scripted stand-in for the models, so it needs no API key.

Results

On real runs against Token Factory, all four demo repos were **fixed on the first attempt**:

Demo	Bug	Result
Inventory refactor	A renamed field plus swapped positional arguments across three modules (the second bug only shows up on bulk orders)	Fixed by Super in 15–31 s
Invoice service	Tax applied before the discount and rounded per line, so totals drift by cents	Fixed by Super in 19–21 s
API rate limiter	Token-bucket refill truncates fractional tokens and never caps at capacity	Fixed by Super in 28 s, and by Ultra in 10–11 s
Python upgrade	Release tooling breaks on Python 3.12+ because `distutils` and `inspect.getargspec` were removed (the second only appears after the first is fixed)	Fixed by Super in 29 s on the live site, with Tavily research and 5 cited sources

Every patch was minimal and correct, every PR write-up named the real root cause, and no attempt tried to edit the tests.

What Tavily changed, measured on the Python upgrade demo:

Run	Search query	Fix	Sources in the PR
Without Tavily	none	Clean switch to `packaging.version` and `inspect.signature`, in 16 s	none
With Tavily, first version of the prompt	"Python 3.14 distutils module removal alternative"	Worse: kept `distutils` behind a `try/except` fallback	5
With Tavily, revised prompt (live site)	"distutils module removed Python 3.12 alternative"	Clean switch to `packaging.version` and `inspect.signature`, in 29 s	5, including PEP 632 and the Python docs

These removals are well known, so Super fixed the bug even without research. On this demo, Tavily's value was a fix **backed by sources a reviewer can check**, rather than a fix that wouldn't otherwise have happened. It costs about 5 s per search, and searches only run for failures that point outside the repo.

Challenges we ran into

- **Nano returned empty JSON.** Nemotron 3 Nano reasons by default, and the reasoning counts toward `max_tokens`. With a 1500-token cap it spent the whole budget thinking and returned nothing, with `finish_reason=length`. Turning thinking off for Nano's structured calls fixed it: 137 tokens of valid JSON in 1.0 s, down from 6.0 s.
- **Web research steered the fix the wrong way at first.** Our first prompt told the agent to "prefer the fix the research recommends". Tavily's answer mentioned compatibility workarounds, and Super copied them, keeping the removed `distutils` behind a `try/except` fallback. Rewording the prompt to treat research as background, and to ask for a clean, forward-compatible fix, got the direct fix back.

- **Trusting the agent.** The agent's own "tests pass" can't be relied on, so verification runs outside it. We also had to stop an agent from making CI pass the easy way, which is why test protection is enforced in two places.
- **Deployment quota.** Our default Nebius region had no CPU quota at all, and we only found out at the last step of deployment. We added project and region targeting to the deploy script and deployed to a region with capacity.
- **Real-world model output.** Models wrap JSON in prose and code fences and put raw newlines inside strings, so the parser had to be tolerant of all three.

Accomplishments that we're proud of

- **Every fix is checked.** AutoFixAI's output isn't "the model says it's fixed". It's a diff that passed your real test suite on a clean run.
- **PRs that cite real sources.** When AutoFixAI researched a failure, the PR lists exactly what Tavily retrieved, so a reviewer can check the reasoning instead of trusting it.
- **Routing across the Nemotron family that clearly pays off.** The expensive model is only paid for when it's needed, and the dashboard shows it.
- **It goes from an empty folder to deployed** on Nebius Serverless, with a live demo, a CLI, a GitHub Action and a test suite, all within the hackathon.

What we learned

- **For agents, reasoning control matters as much as model choice.** The same Nano model went from unusable to excellent for structured calls just by turning thinking off.
- Mid-size models with good tool calling (Nemotron Super) handle most real bugs on their own. The biggest model is best kept for escalation.
- Guardrails belong **outside** the model: in the tool layer and in independent verification, not in the prompt.
- **How retrieved context is framed matters as much as what's retrieved.** The same Tavily results produced a worse fix or a clean one, depending only on whether the prompt treated them as instructions or as background.

What's next for AutoFixAI

- **More languages and test runners:** JavaScript/TypeScript (Jest, Vitest), Go and Rust.
- **Per-job container sandboxes** on Nebius Serverless Jobs, so many fixes can run in parallel.
- **Learning from merged PRs:** feed what reviewers changed back into the triage and reflection prompts.
- **Flaky-test detection:** re-run before fixing, and label likely flakes instead of "fixing" them.
- **Research where it changes the outcome:** test on library releases newer than the models' training data, which is where web research should matter most. Rank official docs and changelogs above forum posts.